Technical report

Academic Paper Analyzer

Technical documentation supporting the thesis:
architecture, design choices and analysis pipeline

Riccardo Aldrovandi

Repository: paper_analyzer

September 2026

Contents

1. Overview 3
2. `main.py` — the orchestrator 3
    2.1 Support functions 3
    2.2 The six subcommands 4
3. `src/config.py` — configuration and system prompt 5
4. `src/api/` — metadata search 5
5. `src/download/downloader.py` — actual download 5
6. `src/processing/` — text and image extraction 6
7. `src/advanced/` — the visuo-mathematical pipeline 6
8. `src/llm/` — the three LLM providers 7
9. `src/utils/` — cross-cutting utilities 7
10. End-to-end flow: `full-analyze` and `export-dataset` 8
11. Pipeline robustness: a cross-cutting summary 8
12. Main dependencies (`requirements.txt`) 9

This report describes in detail how the project's code is built and how it works, file by file. It is meant as supporting documentation for the thesis: it explains what each component does, why it was designed that way, and how the pieces fit together in the overall pipeline.

---

1. Overview

The project is a command-line framework (`main.py`) that automates the whole cycle of a scientific literature review:

1. **Searches** for papers on Semantic Scholar and INSPIRE HEP, and downloads their PDFs and LaTeX sources from arXiv.
2. **Extracts** a paper's text (from the PDF or from the LaTeX source) and its figures/charts.
3. **Digitizes** each chart with a vision model (Gemini Vision, with an OpenRouter fallback), separating overlapping curves into distinct series.
4. **Discovers the mathematical formula** behind each continuous curve via symbolic regression (PySR), validating each equation on a subset of points never seen during the search, to discard formulas that chase the noise instead of describing the real trend.
5. **Enriches** the paper's text with these visual findings, keeping them explicitly separate from the results stated by the authors.
6. **Queries an LLM** (Gemini, Groq or OpenRouter) to produce a structured JSON summary of the paper (contributions, methodology, results, limitations, etc.).
7. **Exports** all of this as a dataset in chat-SFT format, meant for fine-tuning a local model (Qwen/Llama) that will one day replace the paid API calls.

The code is organized by separation of concerns: each folder under `src/` handles a single aspect of the pipeline, and `main.py` acts as the orchestrator that chains them together through the CLI's six subcommands.

```
paper_analyzer/
├── main.py                    # CLI entry point, orchestration of the 6 subcommands
├── src/
│   ├── api/                   # metadata search on Semantic Scholar and INSPIRE HEP
│   ├── download/               # actual download of PDFs and LaTeX sources
│   ├── processing/              # text and image extraction from PDF/LaTeX
│   ├── advanced/                # chart digitization + symbolic regression
│   ├── llm/                    # clients for the three LLM providers (Gemini, Groq, OpenRouter)
│   ├── utils/                   # generic helpers (sanitization, export, retry)
│   └── config.py                # API keys, paths, system prompt for the LLM
└── requirements.txt
```

---

2. `main.py` — the orchestrator

It is the program's entry point and contains both the CLI definition (via `argparse`) and three support functions shared by several commands, to avoid duplicating the same logic in different places.

2.1 Support functions

`run_llm_providers(text_content, model_arg)` Centralizes the "send the text to one or more LLMs and print the resulting JSON" loop. If `model_arg == "all"` it runs on Gemini, Groq and OpenRouter in sequence, otherwise only on the requested provider. It is shared by `analyze` and `full-analyze`, so the two commands return their results in exactly the same format.

`analyze_figure(image_path, iterations)` The core of the visuo-mathematical pipeline applied to a single image. It calls `digitize_plot()` to turn the chart into structured data; if the chart is continuous, for each detected curve it calls `discover_formula()` (PySR) and builds a line of text with the equation found and its reliability metrics (R^2 on the fit and on the holdout, with an explicit warning if the gap between the two

is too large — a sign of overfitting). If the chart is discrete, it instead reports the text summary and the point→label map produced by the digitizer. Any error (unreadable image, a curve that doesn't converge, etc.) is logged and the figure is simply skipped: one broken figure must not make the analysis of the whole paper fail.

`build_enriched_paper_text(pdf, latex, iterations, max_figures)` Puts together the whole "visual" side of the pipeline: it extracts the paper's text, extracts the figures, analyzes them one by one with `analyze_figure()`, and finally appends to the text an `--- ADVANCED PLOT ANALYSIS ---` section with an explicit disclaimer recalling that those formulas are **not** part of the original paper but independent findings of the algorithm. The `max_figures` parameter caps the number of figures actually analyzed per paper: without this limit, a paper with dozens of repetitive charts (typical of appendices with ablation studies) can exhaust the daily API quota on its own. This function is used by both `full-analyze` and `export-dataset`, so the two commands can never behave differently on the same input.

`already_exported_sources(output_path)` Reads an existing JSONL dataset and returns the set of source files already processed, simply by reading the `source_file` field of each line. It is the mechanism that makes `export-dataset` **resumable**: if you interrupt it or run it again, it doesn't re-spend API calls on papers already done.

2.2 The six subcommands

Command	What it does	Does it touch the visual pipeline?
fetch	Searches for papers on Semantic Scholar/INSPIRE, downloads the PDF (always) and LaTeX (if --latex)	No
analyze	Extracts only the text of a single paper and sends it to the LLM	No
images	Extracts only the figures of a single paper	Extraction only, no analysis
advanced	Digitizes a single already-extracted chart and finds its formula	Yes (on its own)
full-analyze	Full pipeline on a single paper: text + figures + formulas + LLM	Yes
export-dataset	Like full-analyze, but in batch on all the downloaded papers, saving inputs/outputs as a JSONL dataset	Yes

`fetch` queries `fetch_semantic_scholar()` and/or `fetch_inspire_hep()` depending on `--api` (`semantic`/`inspire`/`both`), applies an optional filter on the minimum number of citations (`--min-citations`) and, for INSPIRE only, a sort order (`--sort mostrecent|mostcited`) that is passed directly as the `sort` parameter of INSPIRE's REST API. The results are saved as JSON and CSV via `export_results()`.

`analyze` is the "light" version: it extracts the text with `extract_text_from_pdf`/`extract_text_from_latex_tarball` and sends it straight to `run_llm_providers()`, without ever touching images or PySR.

`images` exposes figure extraction (`extract_images_from_pdf`/`extract_images_from_latex`) on the command line, useful to check by eye what the program manages to isolate from a paper before launching the actual analysis.

`advanced` is the test bench of the visuo-mathematical pipeline on a single already-extracted image: it calls `digitize_plot()` and then, for each continuous curve found, `discover_formula()`, printing the equation, its complexity and the two R^2 metrics with the overfitting warning when relevant.

`full-analyze` and **`export-dataset`** are described in detail in section 6 (end-to-end flow), since they are the ones that bring all the modules into play at the same time.

---

3. `src/config.py` — configuration and system prompt

A small but central file. It loads the four API keys from `.env` (`S2_API_KEY`, `GEMINI_API_KEY`, `GROQ_API_KEY`, `OPENROUTER_API_KEY`) via `python-dotenv`, defines the download destination directories (`downloaded_pdfs/`, `downloaded_latex/`, created automatically if missing), and above all contains `SYSTEM_PROMPT`: the prompt prepended to each paper's text in every LLM call.

The prompt defines the JSON schema the LLM must return (10 fields: problem addressed, contributions, methodology, datasets used, experiments, results, limitations, future work, relevant elements, and `visual_analysis_findings`) and contains an explicit and rather insistent instruction that everything found in the text's `--- ADVANCED PLOT ANALYSIS ---` section (the formulas discovered by PySR) **must never be confused with the paper's results**: it must only go in the dedicated `visual_analysis_findings` field, never in `results` or `methodology`. This rule was added after observing empirically that, without a separate field in the schema, the LLM tended to attribute to the paper's authors formulas that were actually approximations made by the algorithm (see section 8).

---

4. `src/api/` — metadata search

`semantic_scholar.py` — **`fetch_semantic_scholar(query, limit, fetch_latex, min_citations)`**
Queries the public endpoint `api.semanticscholar.org/graph/v1/paper/search`. It handles rate limiting robustly: if it gets a 429 it retries up to 5 times with exponential backoff (3^attempt seconds). For each result it extracts the title, authors, year, citations, ArXiv ID (inside `externalIds`) and open-access PDF link; if `min_citations` is set, it discards papers below the threshold right away, before even trying to download them (avoiding useless downloads). The actual download is delegated to `download_pdf()`/`download_latex()` in `src/download/downloader.py`.

`inspire_hep.py` — **`fetch_inspire_hep(query, limit, fetch_latex, min_citations, sort_by)`**
The same logic applied to the INSPIRE HEP API (`inspirehep.net/api/literature`), specialized in high energy physics. It additionally supports the `sort_by` parameter (`mostrecent`/`mostcited`), forwarded directly as the `sort` parameter of the REST query — useful because on INSPIRE a generic topic search tends to return the most cited historical/foundational papers first rather than recent ones, and conversely without a high `--min-citations` you risk picking up poorly relevant results. Here the ArXiv ID is in `metadata.arxiv_eprints`, and it is **not always present**: many papers indexed by INSPIRE are older than arXiv itself (started in 1991) and therefore have no preprint to download — it is a limitation of the data, not a bug (verified empirically, see section 8).

---

5. `src/download/downloader.py` — actual download

Two symmetric functions, both with the same defensive pattern: if the file already exists locally, they skip the download (`export-dataset` and repeated `fetch` runs rely precisely on this to be idempotent).

`download_pdf(pdf_url, paper_id, title)` downloads the PDF with a browser User-Agent (some academic servers block requests without one) and `verify=False` to get past the expired certificates typical of some university servers. It only saves if the response's `Content-Type` is actually `application/pdf`.

`download_latex(arxiv_id, paper_id, title)` downloads the source from `arxiv.org/e-print/<id>`. A check was added here at a later stage: arXiv sometimes responds with a PDF instead of a real gzip archive, when the author didn't upload a separate LaTeX source. The function checks both the `Content-Type` and the first two bytes of the content (`\x1f\x8b`, gzip's magic signature) before saving the file as `.tar.gz`: if it isn't really a gzip archive, it honestly returns `"N/A"` instead of saving a file with a misleading name that would later fail silently during extraction (see section 8, it was a real bug already observed on a concrete case).

---

6. `src/processing/` — text and image extraction

`pdf_extractor.py` — `extract_text_from_pdf(pdf_path)` The simplest of the three: it opens the PDF with `PyMuPDF` and concatenates the text page by page (`page.get_text("text")`). No handling of complex layouts or OCR — if the PDF is a scan without selectable text, the result will be empty or nearly so.

`latex_extractor.py` — `extract_text_from_latex_tarball(tar_path)` It opens the archive with the `tarfile` module in `r:gz` mode, and for each file ending in `.tex` reads its content (trying UTF-8 first, then latin-1 as a fallback), concatenating it with a header that gives the name of the source file. Opening the archive is entirely wrapped in a `try/except`: if the file is not a real gzip archive (see the bug above), the function doesn't raise exceptions upwards but prints an error and returns an empty string — a silent but safe behavior, absorbed further downstream by the "skip if the text is empty" check in `export-dataset`.

`image_extractor.py` — the most elaborate of the three modules, because it doesn't just extract but applies a smart filter.

- `extract_images_from_pdf(pdf_path)`: for each page of the PDF, it first extracts the directly embedded raster images (discarding those under 200×200 pixels, typically logos/icons). Then, if the page mentions "fig"/"figure"/"plot" in its text, it renders a full-page image and passes it to `_is_actual_plot()`: a Gemini Vision call that answers YES/NO to the question "does this page really contain a chart with numerical data, or is it just text/tables/architecture diagrams?". Only if the answer is positive does the page go on to `_crop_individual_plots()`, which asks Gemini Vision for the bounding boxes of each distinct chart on the page (in normalized 0-1000 coordinates) and crops them individually at high resolution (300 DPI), removing the full-page render once the crop has succeeded.
- `extract_images_from_latex(tar_path)`: more direct, because LaTeX sources already contain the figures as separate files (`.png`, `.jpg`, `.pdf`, `.eps`); vector formats are converted to PNG at 300 DPI. It doesn't apply the Gemini filter, because every extracted image is by definition a figure of the paper (not a full page to be judged).

An important design detail: in the `_is_actual_plot()` function, if the Gemini quota is exhausted, the code does **not** delegate the decision to the weaker fallback model (OpenRouter) — it simply skips the filter and keeps the page. The reasoning: here a false "NO" discards the figure forever and silently, while a false "YES" only costs one more attempt downstream (which has its own safe fallback anyway). It is an asymmetric safety choice, added after observing in practice that the free fallback model wrongly labeled as "not a chart" an image that actually was one (see section 8).

---

7. `src/advanced/` — the visuo-mathematical pipeline

`digitizer.py` — `digitize_plot(image_path)` It sends the image to Gemini Vision with a prompt asking it to: (1) classify the chart as continuous or discrete; (2) if continuous, identify **each distinct curve** (by color/legend) and extract 15-30 (X, Y) pairs for each one, never mixing points from different curves; (3) if discrete, extract the individual points with their respective text labels. This is a critical point of the project: the first version asked to extract points from the "main curve" in the singular, and when a chart had several overlapping lines (e.g. the comparison between three variants of a model) the result was a "Frankenstein" set of coordinates mixed across different curves, which PySR then interpolated producing meaningless equations.

The points of each curve are cleaned up by `_clean_points()`: sorted by X and deduplicated (keeping the first occurrence), because out-of-order or duplicate coordinates confuse symbolic regression more than having a few more points helps.

If the Gemini quota is exhausted, the function switches to `analyze_image_with_openrouter()` (defined in `src/llm/openrouter_client.py`) as a fallback, reusing the same prompt.

`pysr.py` — `discover_formula(X, y, iterations)` Applies the actual symbolic regression via the PySR library (Julia engine). Some non-obvious configuration choices, all arising from problems observed during testing:

- **Operators limited** to `+ - * /` and `exp`/`log` (no `sin`/`cos`): with only 15-30 noisy points (approximate readings of a chart), high-frequency trigonometric functions always find a way to "pass close" to the noise, producing a perfect but meaningless fit.
- **`nested_constraints`** forbids nesting `exp`/`log` inside themselves (no `exp(exp(exp(...)))`): even without trigonometry, PySR can stack exponentials to create a "bump" that chases the noise instead of describing a real trend.
- **Holdout validation** (`_select_by_holdout`): with at least 8 points available, a part of them (~25%, at least 3, always leaving at least 5 for the fit) is kept out of the search. Every equation on PySR's Pareto front (not just the "best" one according to its internal score) is then evaluated on these never-seen points; among the equations with a holdout score close to the maximum, the simplest one is chosen (Occam's razor). It is the automated equivalent of a human comparing by eye the curve found with the original chart, and in practice it made it possible to automatically discard formulas that had an R^2 close to 1 on the training data but that, looked at critically, were clearly overfit (e.g. a Gaussian "bump" where the chart showed a monotonic trend).
- **`_r2_score()`** explicitly handles the case of an almost constant target (e.g. a flat reference curve), where the standard R^2 formula is a 0/0 division: without this special case, a perfect fit of a constant was wrongly reported as `nan`/`-inf`.
- **`temp_equation_file=True`** prevents PySR from leaving on disk an `outputs/<timestamp>/` folder for every single curve analyzed (with `export-dataset` launching dozens of them per run, the disk quickly filled up with files that were never read again).

8. `src/llm/` — the three LLM providers

All three expose the same signature `analyze_with_<provider>(text_content) -> tuple[str, str]` (name of the model used, raw JSON string), so `run_llm_providers()` in `main.py` can treat them interchangeably.

`gemini_client.py` uses the `gemini-3.6-flash` model via the `google-genai` SDK. It is the only one implementing full resilience: the call goes through `call_gemini_with_retry()` (see `src/utils/retry.py`), and if the free daily quota is exhausted (HTTP 429) it automatically switches to `analyze_with_openrouter()` as a fallback, so an analysis in progress doesn't stop abruptly just because Gemini has hit its limit.

`groq_client.py` uses the `groq/compound` model. It has two provider-specific operational constraints: it truncates the input text to 20,000 characters (a practical limit of the free plan) and waits 15 seconds before each call to stay within the tokens-per-minute budget.

`openrouter_client.py` is the most elaborate: it exposes both `analyze_with_openrouter()` (text) and `analyze_image_with_openrouter()` (image + prompt, used as the vision fallback by `digitizer.py` and `image_extractor.py`). Both use the same free model alias `openrouter/free` (centralized in the `FREE_MODEL` constant, so the whole project depends on a single definition of "free model"). Both HTTP requests have a 90-second timeout: without a timeout, a slow response from the free tier (which happens, since it is OpenRouter's lowest priority) blocked the script indefinitely instead of failing cleanly and moving on to the next paper.

9. `src/utils/` — cross-cutting utilities

`helpers.py`: `sanitize_filename()` (removes characters that are invalid in file names), `export_results()` (saves a list of dictionaries as JSON and CSV), `process_and_print_result()` (prints the JSON returned by an LLM, handling the case where it isn't valid JSON), `strip_json_fences()` (removes the `json` blocks that the free fallback models sometimes add despite the explicit instruction not to).

`retry.py`: contains `call_gemini_with_retry()` and the `QuotaExceededError` exception. The logic distinguishes two very different kinds of error: a 503 (server temporarily overloaded) is retried up to 5 times with linear backoff (5s, 10s, 15s...), because it is a transient problem; a 429 (daily quota exhausted) is **never** retried — raising `QuotaExceededError` immediately and switching to the fallback is more efficient than waiting in vain, since the quota isn't freed within a few seconds.

---

10. End-to-end flow: `full-analyze` and `export-dataset`

These two commands chain together everything described above:

```
build_enriched_paper_text(pdf/latex)
├─ extract_text_from_pdf / extract_text_from_latex_tarball   → raw text
├─ extract_images_from_pdf / extract_images_from_latex       → list of images
│     (with Gemini Vision filter/crop, PDFs only)
├─ for each image (up to --max-figures):
│     analyze_figure()
│       ├─ digitize_plot()        → (X,Y) points per curve, or discrete points
│       └─ discover_formula()     → equation + fit/holdout R², for each continuous curve
└─ enriched text = original text + "ADVANCED PLOT ANALYSIS" section (with disclaimer)

run_llm_providers(enriched text)  /  direct call to the providers (export-dataset)
└─ analyze_with_gemini/groq/openrouter(enriched text)
→ structured 10-field JSON, with visual_analysis_findings kept separate from the paper's results
```

The only difference between the two commands is their scope: `full-analyze` does this on a single file passed on the command line and prints the result on screen; `export-dataset` does it in batch on all the files in `downloaded_pdfs//downloaded_latex/`, saving each (enriched text, LLM answer) pair as a JSONL line in chat-SFT format (`{"messages": [{"role":"system",...},{"role":"user",...},{"role":"assistant",...}]}`), meant to be used directly as a fine-tuning dataset. `export-dataset` is also resumable (it skips papers already in the output) and automatically discards empty or non-JSON answers, so a failure on a single paper doesn't produce junk lines in the dataset.

---

11. Pipeline robustness: a cross-cutting summary

Several resilience mechanisms span multiple modules and are worth looking at together:

- **Retry on transient errors** (503): implemented once in `src/utils/retry.py` and reused by all three Gemini Vision calls (`image_extractor.py`, `digitizer.py`) plus the text analysis call (`gemini_client.py`).
- **Fallback on exhausted quota** (429): automatically switches from Gemini to OpenRouter free, always through the same centralized `FREE_MODEL` alias, at four different points of the pipeline (page filter, figure cropping, chart digitization, final text analysis) — with the single intentional exception of the "is it a chart, yes/no" filter, where the fallback was disabled for the safety reason explained in section 6.
- **Timeout on direct HTTP requests** (OpenRouter): avoids indefinite hangs when the free tier is slow to respond.

- **Granular skipping instead of a full stop**: an image that can't be digitized, a curve on which PySR fails, a paper whose text turns out empty — they are all logged and skipped individually, without ever making the whole `full-analyze`/`export-dataset` command in progress fail.
- **Statistical rather than visual validation**: the choice of the final equation in `pysr.py` doesn't rely on trusting PySR's internal score, but on a real holdout validation — automating the check that would otherwise require a human looking at every chart.

12. Main dependencies (`requirements.txt`)

`requests`/`python-dotenv` for HTTP and configuration; `pymupdf` for PDF parsing; `google-genai` and `groq` as the official SDKs of the two LLM providers (OpenRouter is called via plain `requests`, it has no dedicated SDK); `pysr`, `numpy`, `pandas`, `sympy` for symbolic regression.

Note that the fine-tuning environment (Unsloth, PyTorch, PEFT, etc.) is **not in this file**: it deliberately lives in a separate repository/Python environment (`~/Scrivania/tesi/llm-finetune`), so as not to mix the heavy training dependencies with the light ones of this CLI.